MyPortfolio

LAPORAN PRAKTIKUM APLIKASI MOBILE

MODUL VI: State Management pada Aplikasi ResepKu (setState() dan Provider)

NIM: 2411533019 • Adila Bintang • Kelas: Praktikum Aplikasi Mobile

Flutter / State Management | Modul 6

Repository GitHub

BAGIAN 1: IDENTITAS MAHASISWA & MATA KULIAH

Mata Kuliah	Praktikum Aplikasi Mobile (DIF61145) — 1 SKS, Semester V
Modul Praktikum	Modul 6 – State Management: setState() dan Provider
Mata Kuliah Teori	Aplikasi Mobile (DIF61144) • Pertemuan 9 (LKM-09)

NIM	**2411533019**
Dosen Pengampu	Arifan Rahman, M.Kom. (`arifanrahman@it.unand.ac.id`)
Program Studi / Fakultas	Departemen Informatika — Fakultas Teknologi Informasi, Universitas Andalas

BAGIAN 2: PENDAHULUAN & LANDASAN TEORI

2.1 Latar Belakang & Capaian Praktikum

Pada aplikasi mobile modern, antarmuka pengguna bersifat reaktif terhadap perubahan data internal maupun interaksi pengguna yang dinyatakan dengan formula fundamental `UI = f(state)`. State adalah data yang dapat berubah selama siklus hidup aplikasi dan secara langsung menentukan bagaimana antarmuka ditampilkan ke layar.

Dalam modul praktikum ini, dibangun aplikasi **ResepKu** yang menjadi fondasi berkelanjutan hingga Modul 10. Praktikum ini bertujuan untuk mengeksplorasi dua spektrum manajemen state pada Flutter:

- **Ephemeral / Local State (`setState()`):** State lokal yang hanya dibutuhkan secara spesifik oleh satu widget atau satu halaman saja (contoh: kalkulasi porsi dan kalori pada halaman detail).
- **App / Global State (`Provider`):** State global yang perlu diakses dan dibagikan (*shared*) oleh banyak widget dan halaman yang tidak berada dalam hierarki hierarkis langsung (contoh: daftar resep favorit, badge counter di AppBar, serta tema aplikasi).

Peruntukan	State lokal (*ephemeral*), dipakai oleh satu widget.	App state, dipakai bersama oleh banyak widget dan halaman.
Contoh di ResepKu	Jumlah porsi dan total kalori di halaman detail.	Daftar resep favorit, badge counter, tema gelap/terang.
Tempat Penyimpanan Data	Field di dalam class `State<T>`.	Class model yang meng-extend `ChangeNotifier`.
Notifikasi Pembaruan UI	Memanggil fungsi `setState(() { ... })`.	Memanggil method `notifyListeners()`.
Cara Membaca Data	Akses langsung variabel field di method `build()`.	`context.watch<T>()` di `build()`, `context.read<T>()` di callback, atau widget `Consumer<T>`.

BAGIAN 3: STRUKTUR ARSITEKTUR PROYEK

Proyek `resepku` dirancang secara modular dengan pemisahan tanggung jawab (*separation of concerns*) ke dalam direktori spesifik di dalam `lib/`:

```
lib/
├── main.dart                          # Titik masuk utama &
├── models/
│   └── recipe.dart                    # Data model resep dan
├── data/
│   └── dummy_recipes.dart             # Kumpulan data resep
├── providers/
```

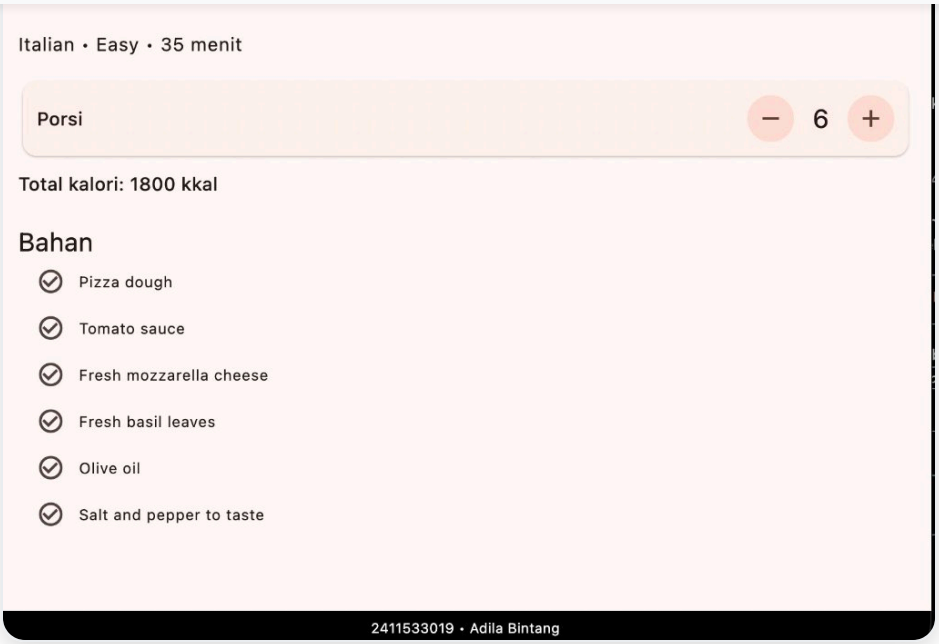
```
│   ├── home_page.dart                # Layar utama katalog
│   ├── detail_page.dart              # Layar detail resep (
│   ├── favorite_page.dart            # Layar daftar resep f
│   └── settings_page.dart            # Layar pengaturan tem
└── widgets/
    ├── identity_footer.dart          # Footer identitas waj
    ├── recipe_tile.dart              # Tile resep responsif
    └── favorite_badge_button.dart    # Tombol AppBar ber-ba
```

BAGIAN 4: CHECKPOINT 1 (STATE LOKAL DENGAN SETSTATE)

4.1 Deskripsi Implementasi

Pada langkah awal (Bagian A), fitur pengaturan porsi resep diimplementasikan di `DetailPage` menggunakan `StatefulWidget`. Jumlah porsi disimpan di variabel privat `int _servings`. Saat pengguna menekan tombol penambah (`+`) atau pengurang (`-`), perubahan dibungkus dengan `setState()` untuk memicu kalkulasi ulang `totalCalories = recipe.caloriesPerServing * _servings` serta merender ulang tampilan secara dinamis.

4.2 Tangkapan Layar Checkpoint 1



Gambar 4.1: Halaman Detail *Classic Margherita Pizza* dengan Porsi bernilai 6 dan Total Kalori 1800 kkal (6 × 300 kkal), lengkap dengan footer identitas `2411533019 • Adila Bintang`.

4.3 Jawaban Pertanyaan Checkpoint 1

> **Pertanyaan Modul:** Tombol `-` tidak aktif ketika porsi bernilai 1. Tunjukkan baris kode yang menyebabkan hal tersebut dan jelaskan mekanismenya!

Potongan Kode (`lib/pages/detail_page.dart`):

```
IconButton.filledTonal(
  onPressed: _servings > 1 ? _removeServing : null,
  icon: const Icon(Icons.remove),
)
```

Penjelasan Analisis:
Pada Flutter, properti `onPressed` pada komponen tombol (seperti `IconButton`, `ElevatedButton`) mengontrol interaktivitas widget tersebut. Kode di atas memanfaatkan ekspresi ternary `_servings > 1 ? _removeServing : null`. Ketika nilai `_servings` berada di

memberikan nilai `null` pada callback interaksi secara otomatis menonaktifkan tombol (*disabled state*), mengubah warna tombol menjadi redup, dan mencegah pengguna mengurangi porsi hingga bernilai nol atau negatif.

BAGIAN 5: PENGAMATAN EKSPERIMEN (BAGIAN B)

Pada Bagian B dilakukan eksperimen menyimpan status favorit secara lokal di dalam widget `RecipeTile` menggunakan `bool _isFavorite = false` dan `setState()`. Berdasarkan pengamatan eksperimen, berikut adalah jawaban atas pertanyaan evaluasi:

❓ 1. Apakah ikon hati berubah? Mengapa?

Jawaban: Ya, ikon hati berhasil berubah bentuk dan warnanya (dari `Icons.favorite_border` abu-abu menjadi `Icons.favorite` merah) saat ditekan. Hal ini terjadi karena variabel `_isFavorite` tersimpan langsung sebagai field di dalam kelas `_RecipeTileState`. Pemanggilan `setState(() => _isFavorite = !_isFavorite)` memberitahu framework Flutter bahwa state internal tile tersebut telah berubah, memicu eksekusi ulang method `build()` pada instance tile bersangkutan dan menampilkan ikon baru.

❓ 2. Coba tampilkan jumlah resep favorit di judul AppBar HomePage. Apakah hal itu dapat dilakukan? Di class mana data _isFavorite berada, dan apakah HomePage dapat membacanya?

Jawaban: Hal tersebut **tidak dapat dilakukan** secara langsung. Variabel `_isFavorite` terisolasi di dalam kelas `_RecipeTileState` yang posisinya berada di bawah (*child widget*) pada pohon hierarki widget. Di Flutter, aliran data bersifat *unidirectional* (turun ke bawah). `HomePage` sebagai *parent widget* tidak memiliki referensi maupun hak akses

❓ 3. Buka halaman detail dari resep yang sudah difavoritkan. Bagaimana halaman detail dapat mengetahui bahwa resep tersebut favorit?

Jawaban: Halaman detail (`DetailPage`) **sama sekali tidak dapat mengetahui** status favorit resep tersebut. Ketika berpindah halaman menggunakan `Navigator.push()`, halaman detail di-push sebagai rute baru yang independen di atas navigator tree. Karena state favorit hanya hidup di memori instance `_RecipeTileState` dan tidak dibagikan (*shared*) ke rute lain, maka halaman detail tidak memiliki sarana untuk membaca status tersebut.

Kesimpulan Eksperimen: State lokal hanya mampu mengendalikan tampilan internal widget itu sendiri. Apabila suatu data diperlukan oleh banyak komponen lintas halaman (daftar resep, badge AppBar, halaman detail, dan halaman favorit), data tersebut harus diangkat (*lifted up*) menjadi App State menggunakan Provider.

BAGIAN 6: IMPLEMENTASI PROVIDER & PENGUJIAN (CHECKPOINT 2)

6.1 Implementasi FavoriteProvider

Package `provider` dipasang melalui perintah `flutter pub add provider`. Dibuat kelas `FavoriteProvider` yang meng-extend `ChangeNotifier`:

```
import 'package:flutter/foundation.dart';
import 'package:resepku/models/recipe.dart';

class FavoriteProvider extends ChangeNotifier {
  final List<Recipe> _items = [];
```

```
  bool isFavorite(Recipe recipe) => _items.any((r) => r.id ==

  void toggle(Recipe recipe) {
    if (isFavorite(recipe)) {
      _items.removeWhere((r) => r.id == recipe.id);
    } else {
      _items.add(recipe);
    }
    notifyListeners(); // Memberi tahu seluruh widget listene
  }

  void clear() {
    _items.clear();
    notifyListeners();
  }
}
```

6.2 Tabel Pengujian Aplikasi (Langkah C.7)

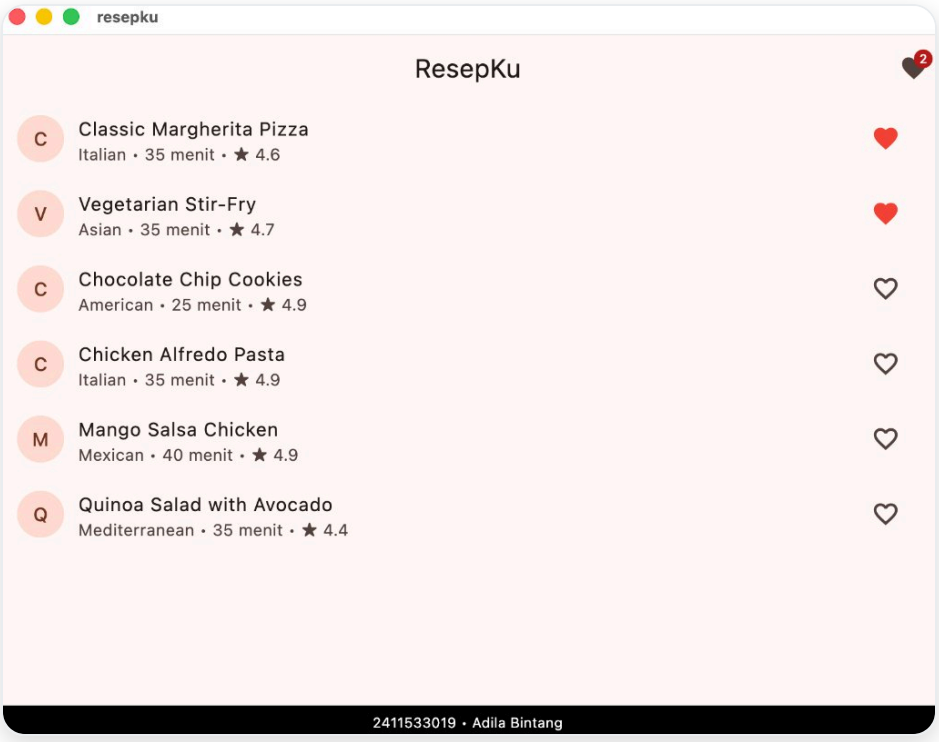
Berdasarkan pengujian keenam skenario yang tercantum pada panduan modul, seluruh fungsionalitas berjalan dengan hasil valid dan konsisten:

No	Skenario Pengujian	Hasil yang Diharapkan	Sesuai?
1	Tap ikon hati pada Margherita dan Alfredo di Home	Ikon berwarna merah, badge menunjukkan angka 2	✓
2	Buka detail Margherita	Ikon hati di AppBar terisi (aktif)	✓
3	Di halaman detail, tap ikon hati, lalu kembali ke Home	Ikon Margherita kosong, badge menunjukkan angka 1	✓

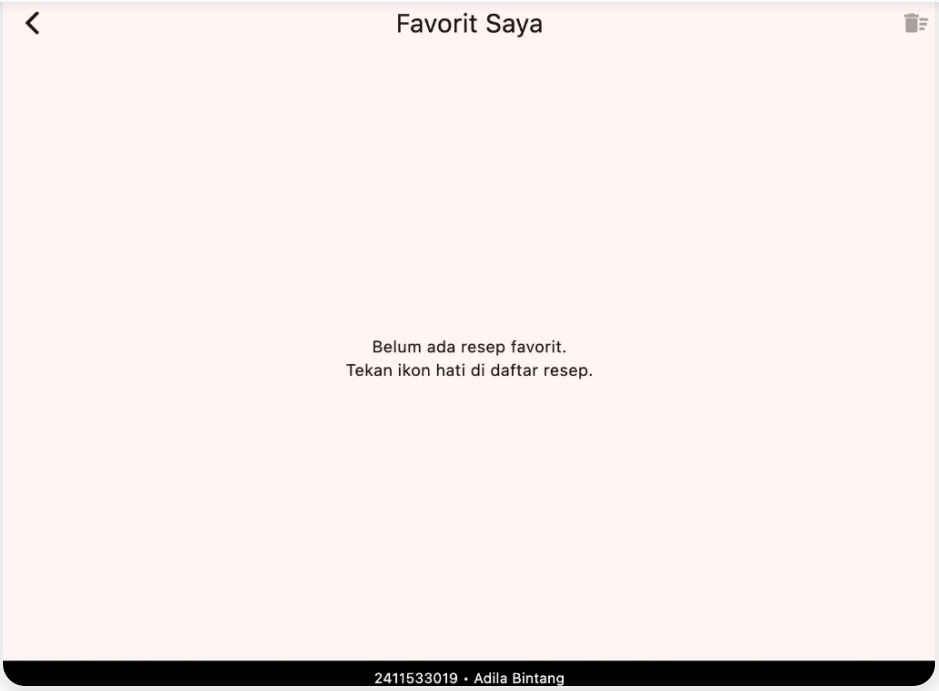
4	Tap badge di AppBar	Halaman Favorit hanya berisi Alfredo	✓
5	Hapus Alfredo dari halaman Favorit, lalu kembali	Muncul pesan "Belum ada resep favorit", badge hilang	✓
6	Ubah porsi menjadi 2, kembali ke Home, lalu buka lagi resep yang sama	Porsi kembali ke nilai awal, karena state lokal ikut dibuang ketika halaman ditutup	✓

6.3 Tangkapan Layar Checkpoint 2

1. HomePage dengan Badge Favorit Bernilai 2



Menampilkan daftar resep di mana Margherita & Stir-Fry difavoritkan, badge di AppBar bernilai 2.



Menampilkan halaman Favorit Saya dengan teks informasi dan tombol hapus semua dalam kondisi disabled.

BAGIAN 7: PERTANYAAN ANALISIS MODUL

1. Rangkuman Pertanyaan Langkah B.2

State lokal (`setState()`) pada `RecipeTile` hanya mampu mengelola dan memperbarui tampilan internal dari widget bersangkutan. Data favorit terisolasi di dalam kelas state lokal sehingga `HomePage` tidak dapat menghitung total item untuk badge `AppBar`, dan `DetailPage` tidak dapat mendeteksi status favorit resep. Hal ini membuktikan bahwa state lokal tidak dapat digunakan untuk mengelola data yang perlu dibagikan (*shared data*) antar-widget atau antar-rute navigasi yang berdiri sendiri.

2. Alasan Porsi Menggunakan setState() dan Bukan Provider (Korelasi Skenario 6)

ruang lingkup kebutuhannya terbatas hanya pada halaman detail resep yang sedang aktif dibuka. Berdasarkan pengujian pada Skenario 6, ketika pengguna meninggalkan rute `DetailPage` dan membukanya kembali, nilai porsi kembali ke nilai default bawaan data resep karena objek `State` tersebut ikut dimusnahkan (*disposed*) dari memori saat halaman ditutup. Menyimpan porsi ke dalam Provider global tidak disarankan karena akan membebani memori dengan data yang tidak dibutuhkan oleh halaman lain seperti `HomePage` atau `FavoritePage`.

3. Analisis Pemindahan ChangeNotifierProvider ke HomePage

Jika `ChangeNotifierProvider` dipindahkan dari fungsi `main()` ke dalam pohon widget `HomePage` (misalnya membungkus `Scaffold` di `HomePage`), maka ketika pengguna menavigasi ke `FavoritePage` melalui `Navigator.push()`, aplikasi akan mengalami crash fatal dengan pesan error `ProviderNotFoundException`. Hal ini terjadi karena rute baru yang dibuka via `Navigator.push()` ditempatkan di tingkat *root* di atas `MaterialApp`, sehingga posisi `FavoritePage` berada sejajar atau di luar cabang sub-pohon `HomePage`. Akibatnya, saat `FavoritePage` mencari instance `FavoriteProvider` ke atas hierarki pohon widget (*ancestor lookup*), provider tersebut tidak ditemukan.

BAGIAN 8: IMPLEMENTASI LATIHAN MANDIRI

8.1 Latihan 1: Tema Gelap dan Terang (Dark & Light Mode)

Menambahkan kemampuan aplikasi untuk beralih antara mode terang dan mode gelap secara global melalui halaman Pengaturan (`SettingsPage`).

a. Kode Provider Tema (`lib/providers/theme_provider.dart`):

MyPortfolio 🏠

```
class ThemeProvider extends ChangeNotifier {
  ThemeMode _themeMode = ThemeMode.light;

  ThemeMode get themeMode => _themeMode;
  bool get isDarkMode => _themeMode == ThemeMode.dark;

  void toggleTheme(bool isDark) {
    _themeMode = isDark ? ThemeMode.dark : ThemeMode.light;
    notifyListeners();
  }
}
```

b. Registrasi MultiProvider & Konfigurasi MaterialApp (`lib/main.dart`):

```
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (context) => FavoriteP
        ChangeNotifierProvider(create: (context) => ThemeProv
      ],
      child: const ResepKuApp(),
    ),
  );
}

class ResepKuApp extends StatelessWidget {
  const ResepKuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ResepKu',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepOrange),
      darkTheme: ThemeData(
        colorSchemeSeed: Colors.deepOrange,
        brightness: Brightness.dark,
      ),
      themeMode: context.watch<ThemeProvider>().themeMode,
      builder: (context, child) => IdentityFooter(child: chil
```

```
}
```

c. Halaman Pengaturan (`lib/pages/settings_page.dart`):

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:resepku/providers/theme_provider.dart';

class SettingsPage extends StatelessWidget {
  const SettingsPage({super.key});

  @override
  Widget build(BuildContext context) {
    final themeProvider = context.watch<ThemeProvider>();
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: ListView(
        children: [
          SwitchListTile(
            title: const Text('Mode Gelap'),
            value: themeProvider.isDarkMode,
            onChanged: (val) {
              context.read<ThemeProvider>().toggleTheme(val);
            },
          ),
        ],
      ),
    );
  }
}
```

MyPortfolio 🏠



Gambar 8.1: Antarmuka Halaman Utama ResepKu saat Mode Gelap aktif (lengkap dengan ikon Pengaturan di AppBar dan footer identitas `2411533019 • Adila Bintang`).

8.2 Latihan 2: Menghapus Semua Favorit dengan Konfirmasi Dialog

Pada halaman `FavoritePage`, ditambahkan aksi pada AppBar berupa tombol `Icons.delete_sweep` untuk menghapus semua item favorit sekaligus. Tombol ini memiliki dialog konfirmasi (`AlertDialog`) sebelum aksi eksekusi pembersihan dilakukan, serta dinonaktifkan secara otomatis (disabled) ketika daftar favorit kosong.

Implementasi AppBar FavoritePage (`lib/pages/favorite_page.dart`):

```
AppBar(
  title: const Text('Favorit Saya'),
  actions: [
```

MyPortfolio 🏠

```
            tooltip: 'Hapus Semua Favorit',
            icon: const Icon(Icons.delete_sweep),
            onPressed: favorite.count == 0
                ? null
                : () {
                    showDialog(
                      context: context,
                      builder: (ctx) => AlertDialog(
                        title: const Text('Hapus Semua Favorit'
                        content: const Text(
                          'Apakah Anda yakin ingin menghapus se
                        ),
                        actions: [
                          TextButton(
                            onPressed: () => Navigator.pop(ctx)
                            child: const Text('Batal'),
                          ),
                          TextButton(
                            onPressed: () {
                              context.read<FavoriteProvider>().
                              Navigator.pop(ctx);
                            },
                            child: const Text('Hapus'),
                          ),
                        ],
                      ),
                    );
                  },
          );
        },
      ),
    ],
  )
```



Gambar 8.2: Tampilan Halaman Favorit Saya saat tidak ada resep favorit. Tombol `delete_sweep` di kanan atas dalam keadaan nonaktif (*disabled*) karena `favorite.count == 0`.

BAGIAN 9: KESIMPULAN

1. **State Lokal (`setState()`)** sangat ideal dan efisien digunakan untuk mengelola data sementara (*ephemeral state*) yang hanya relevan pada satu widget atau rute aktif, seperti fitur penyesuaian porsi dan kalori pada halaman detail.
2. **State Global (`Provider`)** merupakan solusi andal untuk menyelesaikan masalah *prop drilling* dan data terisolasi. Dengan mengabstraksikan state ke dalam kelas turunan `ChangeNotifier` dan menyediakannya di pucuk hierarki widget melalui `MultiProvider`, state dapat dibagikan secara konsisten ke seluruh rute (daftar resep, badge AppBar, detail resep, dan halaman favorit).
3. Pemilihan metode konsumsi state yang tepat—menggunakan `context.watch()` di dalam `build()` untuk mendengarkan perubahan, `context.read()` pada callback event interaktif, dan widget `Consumer` untuk optimasi selective rebuilding—

← Kembali ke Halaman Laporan

Buka Laporan Modul 5

Lihat Repository GitHub

© 2026 MyPortfolio • Adila Bintang (2411533019). All rights reserved.